

INTELIGENCIA ARTIFICIAL Y MINI ROBOTS

ALGORITMOS GENÉTICOS

Desarrollo de tres de los ejercicios propuestos en la sección 3.10

Ejercicio 1. Maximización de una función multimodal

Ejercicio 3. Despacho óptimo de energía eléctrica

Ejercicio 6. Dieta de costo mínimo mediante un algoritmo genético multi-objetivo

Presentado por:

- Edinson Sanchez Fuentes (edsanchezf@unal.edu.co)
- Brayan Alejandro Muñoz Pérez (bmunozp@unal.edu.co)
- Joan Sebastián Roberto Puerto (jroberto@unal.edu.co)

Docente: José J. Martínez P.

12 Septiembre de 2026

Contenido

Contenido	2
1. Introducción.....	3
1.1 Metodología.....	3
2. Ejercicio 1. Maximización de la función $f(x) = x \cdot \sin(10\pi x) + 1$	4
2.1 Planteamiento	4
2.2 Diseño del algoritmo	4
2.3 Estructura del programa.....	5
2.4 Resultados	5
2.5 Verificación	6
2.6 Conclusiones.....	7
3. Ejercicio 3. Despacho óptimo de energía eléctrica	8
3.1 Planteamiento	8
3.2 Primer diseño del algoritmo	9
3.3 Rediseño de la representación	9
3.4 Resultados	10
3.5 Verificación	11
3.6 Conclusiones.....	11
4. Ejercicio 6. Dieta de costo mínimo mediante un algoritmo genético multi-objetivo	13
4.1 Planteamiento	13
4.2 Datos del problema.....	13
4.3 Diseño del algoritmo	14
4.4 Resultados	14
4.5 Verificación	15
4.6 Conclusiones.....	16
5. Análisis comparativo de los tres ejercicios.....	18
5.1 Observaciones de carácter general.....	18
6. Uso de herramientas de IA Generativa	20
6.1 Herramientas empleadas.....	20
6.2 Prompts utilizados	20
6.3 Alcance y limitaciones observadas.....	20
7. Programas, repositorio y ejecución.....	22
7.1 Estructura de los archivos.....	22
7.2 Ejecución en Google COLAB	22
7.3 Ejecución local con Jupyter	22
8. Bibliografía.....	23

1. Introducción

El presente documento contiene el desarrollo de tres de los seis ejercicios propuestos en la sección 3.10 del capítulo Algoritmos Genéticos, conforme a la instrucción del enunciado, que solicita desarrollar tres de los seis. Los ejercicios seleccionados son el primero, el tercero y el sexto.

La selección no fue arbitraria. Los tres ejercicios corresponden a tres tipos distintos de codificación del cromosoma, binaria, real y entera, y a tres secciones diferentes del capítulo: el algoritmo genético simple de la sección 3.3, el tratamiento de restricciones de la sección 3.8 y los algoritmos genéticos multi-objetivo de la sección 3.9. Además, en los tres casos existe una solución exacta calculable por otro método, lo que permite verificar la respuesta obtenida en lugar de limitarse a presentarla.

Este último punto merece énfasis. Un algoritmo genético siempre devuelve una respuesta, puesto que su criterio de parada es habitualmente el número de generaciones y no la optimalidad de la solución. Sin un punto de comparación independiente resulta imposible distinguir un algoritmo que funciona de uno que únicamente produce números con apariencia razonable. Por esa razón cada ejercicio se contrasta contra métodos exactos y se repite con distintas semillas aleatorias.

1.1 Metodología

Los tres programas siguen la misma estructura, que es la propuesta en la sección 3.8 del capítulo bajo el título Diseño de un AG para la solución de problemas:

1. Definir el problema, estableciendo qué se optimiza, qué restricciones deben satisfacerse y por qué el problema justifica el uso de un algoritmo genético.
2. Definir la representación de los individuos, es decir, la forma en que una solución se codifica en un cromosoma y el procedimiento de decodificación correspondiente.
3. Definir los operadores genéticos de selección, cruce y mutación, adecuados al tipo de dato de los genes.
4. Definir la función de aptitud, incluyendo el tratamiento que se dará a las soluciones que violen las restricciones.
5. Definir el criterio de parada y el tamaño de la población.
6. Definir los parámetros restantes, entre ellos la probabilidad de mutación y el seguimiento del mejor cromosoma de cada generación.

2. Ejercicio 1. Maximización de la función $f(x) = x \cdot \sin(10\pi x) + 1$

2.1 Planteamiento

Maximizar la función $f(x) = x \sin(10 \pi x) + 1$, con $x \in [0,1]$.

La función es multimodal: presenta cinco máximos locales dentro del intervalo, uno por cada ciclo del seno, y la amplitud de las crestas crece de forma lineal con la abscisa. Los métodos de optimización basados en el gradiente convergen al máximo local más próximo al punto de arranque, de modo que su resultado depende de la inicialización. El algoritmo genético explora el espacio de búsqueda de forma paralela mediante una población de soluciones candidatas, lo que corresponde al nicho de aplicación que el capítulo describe para la optimización de funciones numéricas.

2.2 Diseño del algoritmo

Paso del diseño	Decisión adoptada
Representación	Algoritmo genético simple. Cromosoma binario de $l = 22$ bits. Decodificación: $x = \text{dec}(\text{cromosoma}) / (2^{22} - 1)$, con paso de 2.384×10^{-7} .
Operadores	Selección por ruleta, cruce en un punto, mutación por bit con $p(m) = 0.02$ y elitismo de un individuo.
Función de aptitud	$\text{apt}(x) = f(x)$, sin desplazamiento. La justificación se expone a continuación.
Criterio de parada	Número fijo de $M = 150$ generaciones.
Población	$K = 60$ cromosomas.

La definición de la función de aptitud requiere una precisión que suele pasarse por alto. La selección por ruleta exige que todas las aptitudes sean estrictamente positivas, pues de lo contrario las probabilidades de selección carecerían de sentido. Ahora bien, antes de aplicar cualquier corrección conviene examinar el recorrido de la función. En el intervalo considerado, el término $x \sin(10 \pi x)$ alcanza su valor mínimo cerca de $x = 0.951$, donde vale aproximadamente -0.9505 , de manera que el mínimo de f en $[0,1]$ es 0.0495 , valor estrictamente positivo. La función puede emplearse entonces directamente como aptitud.

Desplazar la aptitud cuando no es necesario tiene además un costo: al sumar una constante a todos los individuos se reducen las diferencias relativas entre ellos y, con ellas, la presión selectiva de la ruleta. El efecto se cuantificó de forma experimental y los resultados se presentan en la sección 2.4.

2.3 Estructura del programa

El notebook [Ejercicio 1 Maximizar funcion.ipynb](#) implementa las rutinas descritas en la sección 3.5 del capítulo. La rutina genera simula el lanzamiento de una moneda para cada bit y construye la población inicial. La rutina decodifica convierte el genotipo binario en el fenotipo real dentro del intervalo. La rutina evalúa calcula el valor de la función y la aptitud de cada cromosoma. Las funciones `seleccion_ruleta`, `cruce` y `mutacion` implementan los tres operadores genéticos. Finalmente, `algoritmo_genetico` constituye la rutina principal y almacena el histórico del mejor individuo y del promedio de la población en cada generación.

2.4 Resultados

La corrida principal, ejecutada con semilla 2026, produce el cromosoma 1101100111100111100011, que decodifica al valor $x^* = 0.85118958$ con $f(x^*) = 1.85059524$.

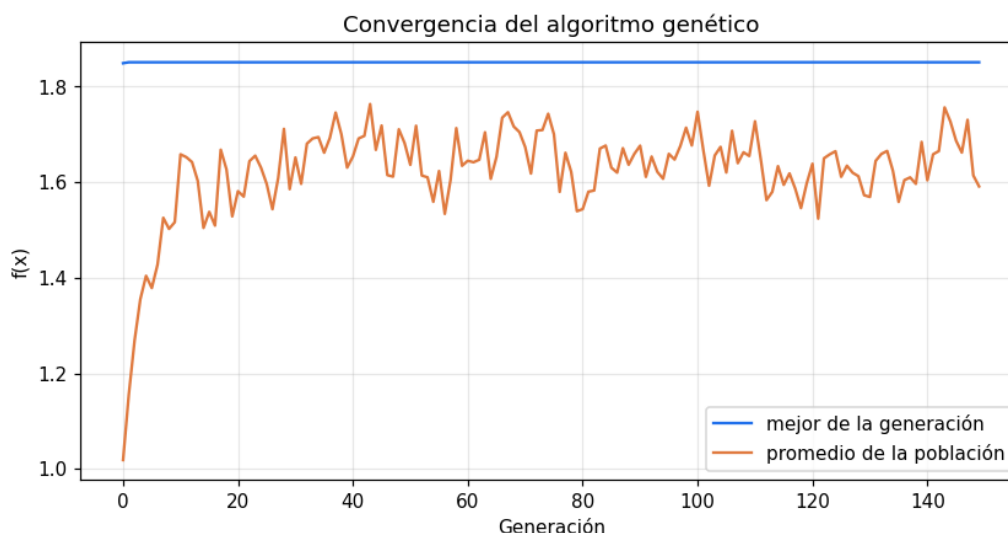


Figura 1. Convergencia del algoritmo. La curva del mejor individuo crece de forma monótona por efecto del elitismo, mientras que el promedio de la población oscila en un nivel inferior, lo que indica que se conserva diversidad.

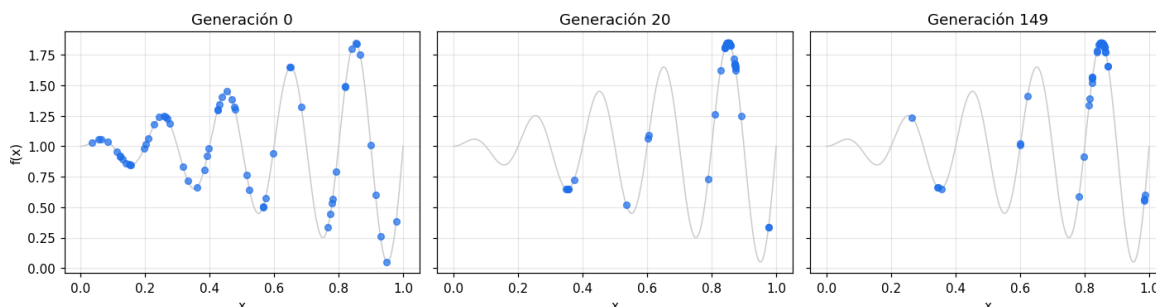


Figura 2. Distribución de la población en el espacio de búsqueda. En la generación inicial los individuos se reparten al azar; hacia la generación 20 se agrupan en las crestas más altas; al final la mayor parte se sitúa sobre el máximo global.

El comportamiento de la Figura 2 ilustra el teorema de los esquemas enunciado en el capítulo. Los bits iniciales que codifican la región cercana a 0.85 constituyen un esquema exitoso que se propaga por la población a través del operador de cruce.

2.5 Verificación

La solución se contrasta con una búsqueda exhaustiva sobre una malla de dos millones de puntos y con un refinamiento numérico local por el método de Brent.

Método	x^*	$f(x^*)$
Búsqueda exhaustiva sobre 2 000 001 puntos	0.85119000	1.85059524
Refinamiento numérico local	0.85118979	1.85059524
Algoritmo genético	0.85118958	1.85059524

El error absoluto en la abscisa es de 2.149×10^{-7} y el error en el valor de la función es de 1.954×10^{-11} . El algoritmo genético empleó 9 000 evaluaciones de la función objetivo frente a los 2 000 001 de la búsqueda exhaustiva, es decir, una relación de 222 a 1.

Dado que el algoritmo es estocástico, el resultado de una sola corrida no permite concluir nada sobre su confiabilidad. El capítulo señala que los reportes estadísticos son en general iguales para diferentes ejecuciones sobre el mismo problema. Se comprobó esa afirmación repitiendo el experimento treinta veces con semillas distintas.

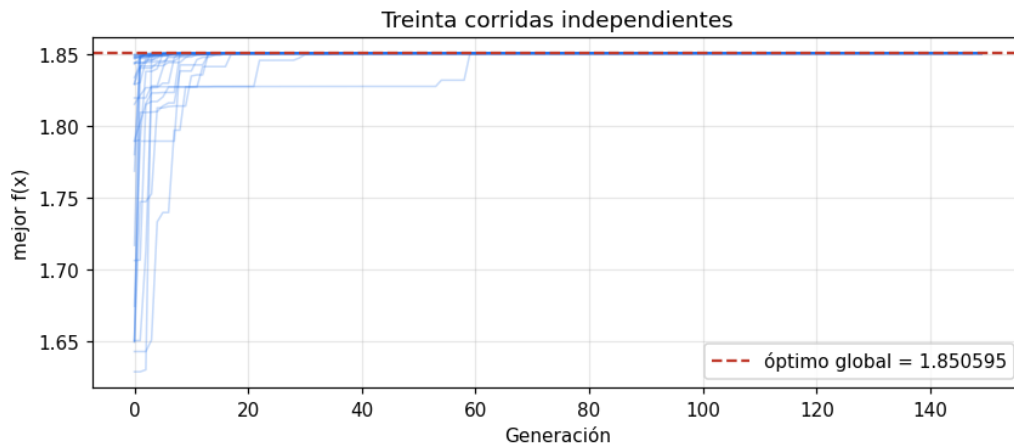


Figura 3. Treinta corridas independientes contrastadas con el óptimo global. Las treinta alcanzan el máximo, con una desviación estándar de 4.28×10^{-9} .

Por último, se verificó de forma experimental la afirmación sobre el desplazamiento de la aptitud formulada en la sección 2.2.

Función de aptitud	Corridas exitosas	$f(x^*)$ medio	Desviación estándar
$\text{apt}(x) = f(x)$	30 de 30	1.85059524	4.28×10^{-9}
$\text{apt}(x) = f(x) + 1$	30 de 30	1.85059524	1.58×10^{-8}

Ambas versiones alcanzan el óptimo global, pero la aptitud sin desplazar presenta una desviación estándar 3.7 veces menor, lo que confirma que conserva mayor presión selectiva.

2.6 Conclusiones

El algoritmo genético localiza el máximo global de la función con un error de 2.1×10^{-7} en la abscisa, empleando 222 veces menos evaluaciones que la búsqueda exhaustiva. La multimodalidad de la función no impide la convergencia, puesto que durante las primeras generaciones la selección por ruleta mantiene individuos distribuidos en varias crestas y la de mayor altura termina por dominar la población.

El elitismo resultó determinante. Sin ese mecanismo, el cruce y la mutación destruyen con frecuencia la mejor solución hallada y la curva de convergencia presenta oscilaciones en lugar de crecer de forma monótona.

El análisis previo del recorrido de la función evitó una corrección innecesaria en la definición de la aptitud y, según mostró el experimento, perjudicial para la presión selectiva. Conviene por tanto verificar el signo de la función objetivo antes de introducir desplazamientos.

Finalmente, la precisión alcanzable queda limitada por la longitud del cromosoma. Con 22 bits el paso mínimo de discretización es 2.384×10^{-7} , valor que coincide con el orden de magnitud del error observado. Aumentar la precisión exige aumentar la longitud del cromosoma y no el número de generaciones.

3. Ejercicio 3. Despacho óptimo de energía eléctrica

3.1 Planteamiento

Una empresa proveedora de energía eléctrica dispone de cuatro plantas de generación para satisfacer la demanda diaria de Cali, Bogotá, Medellín y Barranquilla. Encontrar, usando algoritmos genéticos, el mejor despacho de energía minimizando los costos de transporte y generación.

Tabla 1. Capacidad y costo de generación de las plantas.

Planta	Capacidad (GW/día)	Costo de generación
Planta C	3	680
Planta B	6	720
Planta M	5	660
Planta Ba	4	750

Tabla 2. Costos de transporte entre plantas y ciudades, y demanda de cada ciudad.

Costo de transporte	Cali	Bogotá	Medellín	Barranquilla
Planta C	1	4	3	6
Planta B	4	1	4	5
Planta M	3	4	1	4
Planta Ba	6	5	4	1
Demanda (GW/día)	4	3	5	3

Sobre las unidades es necesaria una precisión. En el enunciado la tabla de generación está expresada en pesos por kilovatio hora, mientras que la de transporte se refiere a cada gigavatio. Para que ambas cantidades resulten sumables se toman las dos en pesos por kilovatio hora, lo cual corresponde a órdenes de magnitud realistas en pesos colombianos: entre 660 y 750 pesos por kilovatio hora de generación y entre 1 y 6 pesos de peaje de transmisión según la ruta. El costo diario se obtiene multiplicando por los 24 millones de kilovatios hora que contiene un gigavatio día. La estructura del problema y su solución óptima no dependen de esta escala se encuentra esta información en el Notebook: [Ejercicio3_Despacho_de_energia](#).

La capacidad total instalada asciende a 18 GW y la demanda total a 15 GW, de manera que el sistema se encuentra sobreofertado. Además de repartir la energía, el algoritmo debe decidir qué capacidad permanece ociosa. Se trata de un problema de transporte con costos de generación, perteneciente a la familia de los problemas de asignación mencionados en la sección 3.6 del capítulo.

3.2 Primer diseño del algoritmo

La solución es una matriz de despacho de cuatro por cuatro cuyas entradas indican la energía que cada planta envía a cada ciudad. Esa matriz debe satisfacer la demanda de cada ciudad de forma exacta y no exceder la capacidad de ninguna planta.

Paso del diseño	Decisión adoptada
Representación	Dieciséis genes reales que representan la prioridad de cada planta para cada ciudad. La demanda de cada ciudad se reparte en proporción a esas prioridades, de modo que la restricción de demanda se satisface por construcción.
Operadores	Selección por torneo de tamaño 3, cruce BLX- α con $\alpha = 0.4$ y mutación gaussiana con desviación decreciente de 0.35 a 0.01, que corresponde a la mutación variable de la sección 3.8.6. Elitismo de tres individuos.
Función de aptitud	Costo total con penalidad moderada por exceso de capacidad, igual a tres veces el mayor costo unitario multiplicado por el exceso.
Criterio de parada	Número fijo de $M = 600$ generaciones.
Población	$K = 150$ cromosomas.

El tratamiento de la restricción de capacidad merece comentario. La sección 3.8 del capítulo plantea tres enfoques posibles ante las soluciones infractoras: la eliminación, la penalidad alta y la penalidad moderada. Se adoptó la penalidad moderada, que es la misma que el capítulo emplea en el problema del furgón, donde la penalidad es un múltiplo del exceso de peso. Su ventaja consiste en que no elimina al infractor sino que reduce su aptitud, de manera que un cromosoma que viole ligeramente la capacidad puede seguir aportando material genético útil a las generaciones siguientes.

El resultado de esta primera versión fue insatisfactorio. En diez corridas independientes el algoritmo se situó en promedio a 0.168 % del óptimo y ninguna de ellas alcanzó la solución exacta.

3.3 Rediseño de la representación

La causa del comportamiento anterior no reside en el número de generaciones ni en el tamaño de la población, sino en la representación escogida. El óptimo de un problema de transporte se encuentra siempre en un vértice del poliedro de soluciones factibles, lo que implica que muchas de las variables valen exactamente cero y que las plantas utilizadas trabajan a plena capacidad. La decodificación proporcional, en cambio, reparte la demanda de cada ciudad entre todas las plantas, de modo que alcanzar un cero exacto exigiría que una prioridad se anulara por completo, situación que la mutación gaussiana prácticamente nunca produce.

Siguiendo la advertencia del capítulo sobre la necesidad de volver a pasos anteriores del diseño, se regresó al paso segundo y se sustituyó la decodificación proporcional por una decodificación voraz. En ella, dieciséis genes representan la prioridad de cada planta para cada ciudad y cuatro genes adicionales definen el orden en que se atienden las ciudades. Cada ciudad toma su demanda de las plantas en orden de prioridad descendente hasta agotar la capacidad disponible de cada una.

Como la capacidad total supera a la demanda total, esta decodificación produce siempre soluciones factibles, lo que elimina la necesidad de la penalidad, y produce además soluciones situadas en vértices, que es precisamente donde se encuentra el óptimo.

3.4 Resultados

Despacho óptimo (GW/día)	Cali	Bogotá	Medellín	Barranq.	Generado	Capacidad
Planta C, 680 \$/kWh	3	0	0	0	3	3
Planta B, 720 \$/kWh	1	3	0	2	6	6
Planta M, 660 \$/kWh	0	0	5	0	5	5
Planta Ba, 750 \$/kWh	0	0	0	1	1	4

El costo total del despacho óptimo asciende a 250 464 000 000 pesos diarios. La solución asigna la totalidad de la capacidad de las tres plantas más económicas y deja ociosos tres de los cuatro gigavatios de la Planta Ba, que es la más costosa.

Este comportamiento se explica por la estructura de los costos. Las diferencias entre los costos de generación alcanzan 90 pesos por kilovatio hora, mientras que las diferencias entre los costos de transporte no superan los 5 pesos. El costo de generación domina por tanto la decisión de cuánto genera cada planta, y el costo de transporte determina únicamente el detalle de qué planta atiende a qué ciudad. Puede comprobarse en la matriz obtenida: la Planta C entrega sus 3 GW a Cali, que es su destino más económico, y la Planta M entrega sus 5 GW a Medellín por la misma razón.

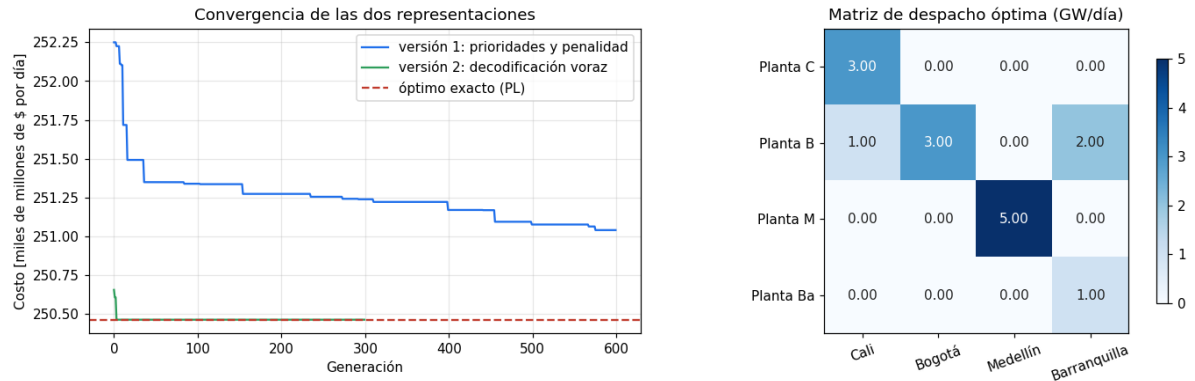


Figura 4. A la izquierda, convergencia de las dos representaciones frente al óptimo exacto. A la derecha, matriz de despacho obtenida con la decodificación voraz.

3.5 Verificación

El problema, tal como está planteado, es un problema de transporte lineal y admite por tanto solución exacta mediante programación lineal. Dado que las capacidades y las demandas son números enteros, el óptimo de la relajación lineal también es entero, lo que permite una segunda verificación independiente por enumeración completa de los despachos enteros posibles.

Método	Costo diario	Diferencia media en diez corridas	Corridas exactas
Programación lineal, método símplex	250 464 000 000	exacto	—
Enumeración de 784 000 despachos enteros	250 464 000 000	exacto	—
Algoritmo genético, primera versión	251 040 881 716	0.168231 %	0 de 10
Algoritmo genético, decodificación voraz	250 464 000 000	0.000000 %	10 de 10

Los dos métodos exactos coinciden entre sí, lo que confirma la corrección de la referencia. La segunda versión del algoritmo genético alcanza ese mismo valor en las diez corridas y, en la ejecución presentada, lo encuentra en la generación 3.

3.6 Conclusiones

El primer resultado del ejercicio es que la representación de los individuos pesa más que los parámetros del algoritmo. La versión inicial se mantuvo a 0.168 % del óptimo en promedio y ninguna corrida alcanzó la solución exacta. Al modificar únicamente la decodificación, conservando los mismos operadores, la misma población y la misma probabilidad de mutación, el algoritmo encontró el óptimo exacto en las diez corridas. El factor limitante no era la capacidad de búsqueda sino la forma de codificar la solución.

La decodificación por prioridades normalizadas asegura el cumplimiento de la restricción de demanda, y la penalidad moderada basta para que la restricción de capacidad se respete sin necesidad de eliminar individuos. La decodificación voraz va más lejos y elimina por completo la necesidad de penalidades, puesto que todo cromosoma resulta factible por construcción. Este es, en general, el enfoque preferible cuando el problema lo permite.

Los operadores reales empleados, el cruce BLX- α y la mutación gaussiana con desviación decreciente, constituyen la adaptación natural del algoritmo genético simple binario a un problema continuo. La reducción progresiva de la desviación resultó necesaria, pues sin ella el algoritmo alcanza con rapidez la región de interés pero no logra afinar los últimos decimales.

Conviene señalar, por último, que para un problema de este tamaño la programación lineal es preferible al algoritmo genético, pues es exacta e inmediata. El algoritmo genético se justifica cuando se incorporan condiciones no lineales, tales como pérdidas dependientes de la distancia, costos escalonados, restricciones de arranque y parada de plantas o contratos de suministro mínimo, que sitúan al problema fuera del dominio de la programación lineal.

4. Ejercicio 6. Dieta de costo mínimo mediante un algoritmo genético multi-objetivo

4.1 Planteamiento

Tome el algoritmo de la dieta y ahora incluya costos. Ahora encuentre una dieta que trate de satisfacer la dieta pero con un costo mínimo. Este es un ejemplo de AG multi-objetivo con dos funciones objetivo.

En el Notebook: [Ejercicio 6 Dieta multiobjetivo](#) el problema de la dieta consiste en determinar cuántas porciones de cada alimento se deben consumir diariamente para cubrir los requerimientos nutricionales de una persona. Al incorporar el costo aparecen dos objetivos en conflicto: minimizar la deficiencia nutricional y minimizar el costo diario de la canasta.

El conflicto es real y no artificial, puesto que los alimentos con mayor densidad nutricional, como el pollo, la leche o el huevo, son también los más costosos por porción. En consecuencia no existe una única solución óptima sino un conjunto de soluciones no dominadas, entendiendo por tales aquellas en las cuales no es posible mejorar la nutrición sin encarecer la canasta, ni abaratarla sin empeorar la nutrición. Ese conjunto es el frente de Pareto descrito en la sección 3.9 del capítulo.

El problema pertenece además a la familia del problema del furgón presentado en la sección 3.7, en el cual los elementos poseen un peso, que aquí es el costo, y un valor de utilidad, que aquí es el aporte nutricional, sujetos a restricciones. Se trata de un problema NP-Completo.

4.2 Datos del problema

Tabla 3. Aporte nutricional y costo por porción de cada alimento.

Alimento (porción)	kcal	Prot. (g)	Calcio (mg)	Hierro (mg)	Vit. C (mg)	Fibra (g)	Costo (\$)
Arroz, taza	205	4.3	16	1.9	0.0	0.6	700
Fríjol, taza	245	15.0	80	3.6	1.5	15.0	1 200
Huevo, unidad	72	6.3	28	0.9	0.0	0.0	750
Pollo, 100 g	165	31.0	15	1.0	0.0	0.0	3 200
Leche, vaso	150	8.0	300	0.1	2.4	0.0	1 400
Banano, unidad	105	1.3	6	0.3	10.3	3.1	900
Papa, unidad	160	4.3	26	1.9	42.0	3.8	600
Lenteja, taza	230	18.0	38	6.6	3.0	15.6	1 100
Zanahoria, unidad	25	0.6	20	0.2	3.6	1.7	400
Pan integral, tajada	69	3.6	30	0.7	0.0	1.9	500

Alimento (porción)	kcal	Prot. (g)	Calcio (mg)	Hierro (mg)	Vit. C (mg)	Fibra (g)	Costo (\$)
Requerimiento diario	2 000	50	1 000	14	60	25	—

4.3 Diseño del algoritmo

Paso del diseño	Decisión adoptada
Representación	Diez genes enteros, cada uno con el número de porciones diarias de un alimento, entre 0 y 4. El espacio contiene $5^{10} = 9\,765\,625$ dietas.
Objetivos	f1, suma de los faltantes relativos respecto a cada requerimiento, que se minimiza. f2, costo diario de la canasta, que se minimiza. Ninguno de los dos se convierte en restricción.
Operadores	Torneo binario por número de frente y, en caso de empate, por distancia de apiñamiento. Cruce uniforme. Mutación mixta con $p(m) = 0.30$, que reemplaza el gen por un valor aleatorio en la mitad de los casos y lo desplaza una porción en la otra mitad.
Algoritmo	NSGA-II, con ordenamiento por no dominancia, distancia de apiñamiento y supervivencia elitista de tipo $(\mu + \lambda)$.
Criterio de parada	Número fijo de $M = 600$ generaciones.
Población	$K = 150$ cromosomas.

Los faltantes se miden en términos relativos y no absolutos para que un déficit de calcio, expresado en miligramos sobre un requerimiento de mil, y un déficit de hierro, expresado en miligramos sobre un requerimiento de catorce, contribuyan con el mismo peso a la función objetivo.

Cuando existen dos objetivos en conflicto no es posible definir una aptitud escalar sin imponer de antemano una ponderación entre ellos, ponderación que sería arbitraria y que ocultaría las demás soluciones de compromiso. Por esa razón se implementó NSGA-II, que consta de tres piezas. La primera es el ordenamiento por no dominancia, según el cual una solución domina a otra si no es peor en ningún objetivo y es mejor en al menos uno; el primer frente lo forman las soluciones no dominadas por ninguna otra, el segundo las que quedan no dominadas al retirar el primero, y así sucesivamente. La segunda es la distancia de apiñamiento, que dentro de un mismo frente privilegia las soluciones situadas en zonas poco pobladas, con el fin de que el frente resultante quede bien distribuido. La tercera es la supervivencia elitista, en la cual padres e hijos compiten conjuntamente.

4.4 Resultados

Antes de la versión multi-objetivo se ejecutó el algoritmo de la dieta en su forma original, es decir, un algoritmo genético que busca únicamente cubrir los requerimientos nutricionales sin considerar el precio. Su resultado sirve de referencia para cuantificar el beneficio de incorporar el segundo objetivo.

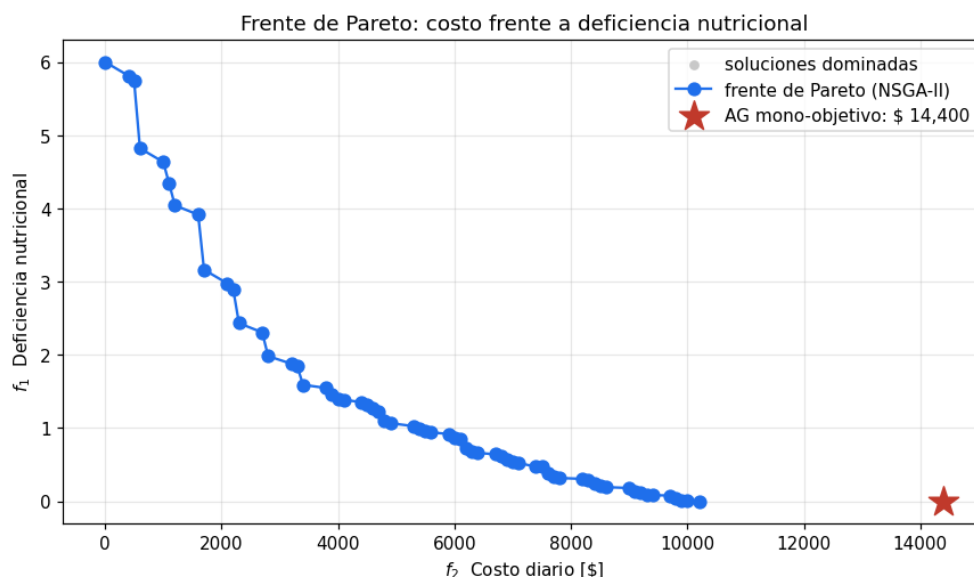


Figura 5. Frente de Pareto obtenido. La estrella señala la solución del algoritmo mono-objetivo, que satisface la dieta pero ignora el costo.

El frente de Pareto no resuelve la decisión por sí solo: entrega el conjunto de compromisos disponibles y corresponde al diseñador escoger uno de ellos en función del contexto, tal como indica el capítulo. Se presentan tres puntos representativos.

Solución del frente	Costo diario	Deficiencia	Composición
Punto de quiebre	3 400	1.590	2 papas y 2 tazas de lenteja
Nutricionalmente completa	10 200	0.000	2 arroz, 4 frijol, 2 leche y 2 papa
Algoritmo mono-objetivo	14 400	0.000	cubre la dieta con un 41 % más de costo

Incorporar el costo como segundo objetivo produce un beneficio cuantificable: la misma cobertura nutricional que alcanza el algoritmo de la dieta original se obtiene con un ahorro de 4 200 pesos diarios, equivalente al 29.2 % del costo y a 1 533 000 pesos anuales.

4.5 Verificación

Como el espacio de búsqueda contiene 9 765 625 dietas, resulta posible calcular el frente de Pareto verdadero por enumeración completa y compararlo con el obtenido por el algoritmo genético. Esta comparación constituye la prueba definitiva sobre la calidad del resultado.

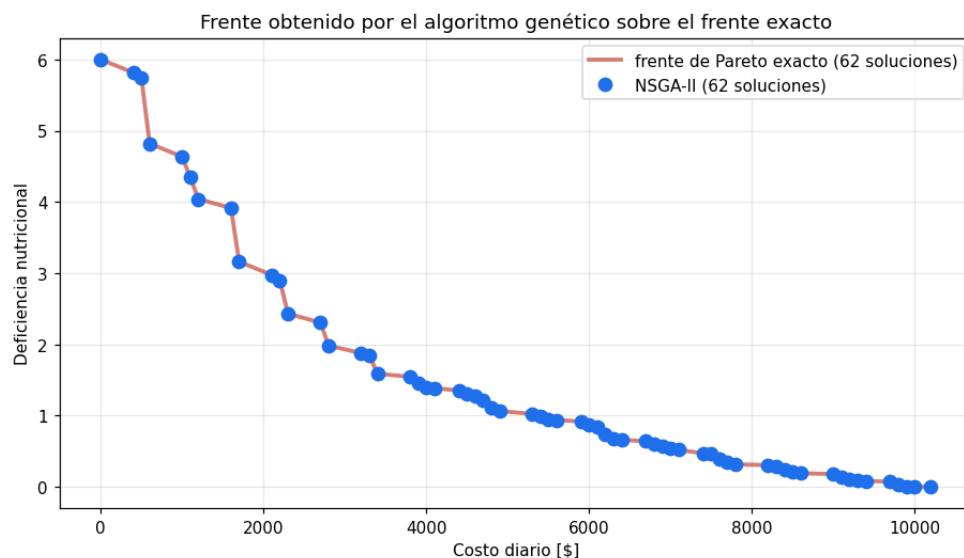


Figura 6. Frente obtenido por NSGA-II superpuesto al frente de Pareto exacto calculado por enumeración completa.

Medida	Resultado
Soluciones del frente de Pareto exacto	62
Soluciones distintas halladas por NSGA-II	62
Soluciones del algoritmo que pertenecen al frente exacto	61 de 62, equivalente al 98.4 %
Dieta completa más económica, óptimo exacto	10 200 pesos diarios
Dieta completa más económica, NSGA-II en la corrida presentada	10 200 pesos diarios
Diez corridas independientes	5 de 10 alcanzan el óptimo exacto; las restantes se detienen en 10 300 pesos, un 0.98 % por encima

El último dato de la tabla constituye el límite del método y conviene exponerlo con claridad. El extremo del frente es la región más difícil para un algoritmo multi-objetivo, precisamente porque la distancia de apiñamiento distribuye el esfuerzo de búsqueda a lo largo de todo el frente en lugar de concentrarlo en un punto. Si el objetivo fuera únicamente hallar la dieta completa más económica, un algoritmo mono-objetivo con el costo como aptitud y la nutrición como restricción resultaría más adecuado. El enfoque multi-objetivo es superior cuando interesa conocer el conjunto completo de compromisos, que es lo que pide este ejercicio.

4.6 Conclusiones

El algoritmo genético multi-objetivo no produce una respuesta única sino un conjunto de respuestas. NSGA-II entregó un frente de dietas no dominadas que va desde la más económica y nutricionalmente deficiente hasta la nutricionalmente completa, lo que hace explícito el precio

de cada punto porcentual adicional de cobertura. Esa información no se obtendría optimizando un solo objetivo.

La verificación contra el frente exacto muestra que el algoritmo recupera el 98.4 % de las soluciones no dominadas, y que en la corrida presentada la dieta completa de menor costo coincide con el óptimo exacto. La prueba de robustez, sin embargo, indica que ese extremo se alcanza solamente en la mitad de las corridas, con una desviación máxima inferior al uno por ciento en las restantes.

La distancia de apiñamiento resultó indispensable para que las ciento cincuenta soluciones no terminaran concentradas en el extremo económico del frente. Sin ese mecanismo el algoritmo converge hacia un solo punto y se pierde justamente aquello que hace útil al enfoque multi-objetivo.

Cabe observar, finalmente, que el frente contiene una solución degenerada de costo cero que corresponde a no consumir nada. Es formalmente no dominada, puesto que ninguna otra dieta es más económica, pero carece de sentido práctico. Ilustra que el frente de Pareto puede contener soluciones matemáticamente correctas que el criterio del diseñador debe descartar.

5. Análisis comparativo de los tres ejercicios

	Ejercicio 1	Ejercicio 3	Ejercicio 6
Tipo de gen	Binario, 22 bits	Real, 16 y 20 genes	Entero, 10 genes
Selección	Ruleta	Torneo de tamaño 3	Torneo binario por frente y apiñamiento
Cruce	Un punto	BLX- α	Uniforme
Mutación	Por bit, $p = 0.02$	Gaussiana variable, $p = 0.15$	Mixta, $p = 0.30$
Restricciones	Ninguna	Penalidad moderada y, luego, codificación factible	Ninguna, por construcción
Referencia exacta	Búsqueda exhaustiva	Programación lineal y enumeración	Frente de Pareto exacto
Resultado	Óptimo global en 30 de 30 corridas	Óptimo exacto en 10 de 10 corridas	98.4 % del frente exacto

5.1 Observaciones de carácter general

La primera observación, y la más importante, es que la representación de los individuos influye más en el resultado que los parámetros del algoritmo. En el ejercicio 3 el problema no se resolvió aumentando la población ni la probabilidad de mutación, sino rediseñando la forma de codificar la solución. El cambio llevó el resultado de una aproximación aceptable a la solución exacta en todas las corridas.

La segunda es que un algoritmo genético siempre entrega una respuesta, y que únicamente la verificación permite determinar si esa respuesta es correcta. Por ese motivo los tres ejercicios se contrastaron contra métodos exactos y se repitieron con distintas semillas aleatorias. Sin ese procedimiento no habría sido posible detectar, por ejemplo, que la primera versión del ejercicio 3 se quedaba sistemáticamente por encima del óptimo.

La tercera es que el manejo de restricciones descrito en la sección 3.8 del capítulo puede evitarse en muchos casos mediante una codificación adecuada. En los ejercicios 3, en su segunda versión, y 6, todo cromosoma resulta factible por construcción, lo que elimina el riesgo de estancamiento por presencia de infractores en la población.

La cuarta es que el algoritmo genético no es siempre la herramienta más adecuada, y señalarlo forma parte del análisis. En el ejercicio 3 la programación lineal es exacta e inmediata. En el ejercicio 6, si el interés se limitara a la dieta más económica, un algoritmo mono-objetivo sería preferible al multi-objetivo. El algoritmo genético muestra su ventaja cuando el problema crece en tamaño o pierde la linealidad.

6. Uso de herramientas de IA Generativa

Los parámetros de entrega del curso establecen que, cuando se utilicen herramientas de IA Generativa, se deben hacer comentarios sobre qué herramientas se emplearon, los prompts utilizados y hasta qué punto sirvieron para el desarrollo de la documentación, los ejercicios o los proyectos. Este capítulo atiende esa exigencia.

6.1 Herramientas empleadas

Herramienta	Uso que se le dio	Utilidad observada
Claude, de Anthropic	Entendimiento y codificación de los tres algoritmos, corrección de errores de los notebooks.	Redujo el tiempo dedicado a corregir errores del código.
Documentación oficial de NumPy, SciPy y Matplotlib	Consulta de la interfaz de programación, en particular de <code>numpy.lexsort</code> y de <code>scipy.optimize.linprog</code> .	Necesaria para resolver detalles que la herramienta de IA resolvía de forma aproximada o incorrecta.
El capítulo del curso	Fuente de la metodología de los seis pasos, de los operadores genéticos, del tratamiento de restricciones y de la sección sobre algoritmos multi-objetivo.	Fundamental. Define el contenido que debía tener cada programa.

6.2 Prompts utilizados

Los prompts más productivos fueron aquellos que incorporaban el contexto del capítulo y que exigían una verificación del resultado, no solamente su obtención. Se transcriben los principales.

«¿Es necesario desplazar la función de aptitud para usar selección por ruleta en este problema? Calcula el mínimo real de la función en el intervalo antes de responder.»

«El AG del despacho de energía se queda a 0.23 % del óptimo de la programación lineal. ¿Por qué ocurre y qué cambio en la representación lo corregiría?»

«Implementa NSGA-II con ordenamiento por no dominancia y distancia de apiñamiento para el problema de la dieta con dos objetivos, y compara el frente obtenido con el frente de Pareto exacto calculado por fuerza bruta.»

6.3 Alcance y limitaciones observadas

La herramienta resultó especialmente útil en cuatro aspectos. El primero es la escritura de código vectorizado con NumPy, gracias al cual los tres programas se ejecutan en segundos porque toda la población se evalúa mediante operaciones matriciales. El segundo es la implementación de algoritmos conocidos pero laboriosos, como NSGA-II o el cruce BLX- α . El tercero, y probablemente el más valioso, es el diagnóstico de las causas por las cuales un algoritmo no converge: la explicación de que el óptimo de un problema de transporte se sitúa

en un vértice del poliedro condujo directamente al rediseño del ejercicio 3. El cuarto es la construcción de las verificaciones independientes, que constituyen la parte del trabajo que con mayor facilidad se omite.

También se identificaron limitaciones que obligaron a corregir el material. Los valores numéricos que la herramienta anticipaba en las conclusiones no coincidían con los que arrojó la ejecución real, de modo que todas las cifras de este documento se tomaron de la salida efectiva de los notebooks y no de lo que la herramienta suponía. Se detectó además un error conceptual en la justificación inicial de la función de aptitud del ejercicio 1, en la cual se afirmaba que la función tomaba valores negativos en el intervalo; el cálculo del mínimo real mostró que no es así y obligó a reformular esa sección. Aparecieron asimismo errores de programación que solo se manifiestan al ejecutar, como un desbordamiento de enteros sin signo al restar dos matrices de imagen.

Por último, las decisiones de modelado, tales como la selección de los alimentos y sus precios, la interpretación de las unidades de los costos del ejercicio 3 o la definición de los requerimientos nutricionales, corresponden al criterio de quien desarrolla el trabajo y debieron tomarse y justificarse de forma explícita.

7. Programas, repositorio y ejecución

7.1 Estructura de los archivos

Los tres programas se entregan como notebooks de Jupyter, ejecutados y con sus resultados incluidos, de acuerdo con el cuarto parámetro de entrega. La carpeta correspondiente del repositorio contiene lo siguiente.

```
3. Algoritmos geneticos/  
├── README.md  
├── requirements.txt  
├── Documento_Algoritmos_Geneticos.pdf  
├── notebooks/  
│   ├── Ejercicio_1_Maximizar_funcion.ipynb  
│   ├── Ejercicio_3_Despacho_de_energia.ipynb  
│   └── Ejercicio_6_Dieta_multiobjetivo.ipynb  
└── img/
```

7.2 Ejecución en Google COLAB

Para abrir un notebook en COLAB ingresar por los hipervínculos de este documento o puede ingresar a colab.research.google.com y se utiliza la opción Archivo, Abrir notebook, GitHub, indicando la dirección del repositorio; también puede subirse el archivo directamente. Una vez abierto se ejecuta mediante Entorno de ejecución, Ejecutar todas. No se requiere instalar nada, puesto que NumPy, SciPy, pandas y Matplotlib vienen preinstalados en ese entorno.

7.3 Ejecución local con Jupyter

```
pip install -r requirements.txt  
jupyter notebook notebooks/
```

Los tres notebooks utilizan semillas aleatorias fijas, declaradas mediante `numpy.random.default_rng`, de manera que los resultados son reproducibles: al volver a ejecutarlos se obtienen exactamente las mismas cifras que aparecen en este documento. Esa reproducibilidad se comprobó reejecutando los tres notebooks desde cero y comparando todas sus salidas, que coincidieron con excepción de los tiempos de ejecución impresos.

8. Bibliografía

- Cardona, D. (2024). P not equal to NP. Universidad Nacional de Colombia. Recuperado de <https://arxiv.org/pdf/2401.15079.pdf>
- Deb, K., Pratap, A., Agarwal, S. y Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182-197.
- Goldberg, D. (1989). *Genetic algorithms in search, optimization and machine learning*. Addison-Wesley.
- Haupt, R. y Haupt, S. (2004). *Practical genetic algorithms*. John Wiley and Sons.
- Holland, J. (1995). *Adaptation in natural and artificial systems* (2.^a ed.). MIT Press.
- Martínez, J. J. (2026). Algoritmos genéticos, capítulo 3. Material del curso de Inteligencia Artificial.
- Martínez, J. J. (2026). Panorama de la IA, capítulo 1. Material del curso de Inteligencia Artificial.
- Mitchell, M. (1999). *An introduction to genetic algorithms*. MIT Press.